42.1 项目规划

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~350k tokens，输出 ~4k tokens（本节）

经过前面 33 章的源码分析，读者已经深入理解了 OpenClaw 的架构和实现细节。本章将从实践角度出发，指导读者构建一个简化版的多通道 AI 助手——**MiniClaw**。MiniClaw 不是 OpenClaw 的完整复制品，而是提取其核心设计思想的**最小可行产品**（MVP）。

42.1.1 需求分析

核心需求

MiniClaw 的目标是实现以下功能：

功能	对应 OpenClaw 模块	MVP 范围
Gateway 服务器	`src/gateway/`	WebSocket 服务器 + RPC 协议
会话管理	`src/sessions/`	内存会话存储 + 基础路由
AI Agent 运行时	`src/agents/`	单 Agent + 工具调用循环
消息通道	`src/channels/`	Telegram Bot + WebChat
工具系统	`src/agents/tools/`	Bash 执行 + 文件读写
记忆系统	`src/memory/`	Markdown 文件记忆

非需求（MVP 排除）

以下功能在 MVP 中不实现，但预留扩展接口：

- 多 Agent 架构
- 沙箱隔离
- Canvas / A2UI
- 浏览器自动化

- Cron 调度
- 原生客户端
- 技能系统

用户故事

```
作为用户，我希望能够：
1. 通过 Telegram 向我的 AI 助手发送消息
2. 通过 Web 界面与 AI 助手交互
3. 让 AI 助手执行本地命令（如查看文件、运行脚本）
4. AI 助手能记住我之前告诉它的事情
5. 在 VPS 上部署助手，随时随地使用
```

42.1.2 架构设计参考

MiniClaw 的架构借鉴 OpenClaw 的分层设计，但大幅简化：

```
┌──────────────┐    ┌──────────────┐
│ Telegram Bot│    │   WebChat    │
└──────────────┘    └──────────────┘
       │                  │
┌──────▼──────────────────▼──────┐
│       Gateway (WebSocket)       │
│   · RPC 协议                    │
│   · 会话路由                    │
│   · 事件广播                    │
└────────────────────────────────┘
                │
┌───────────────▼────────────────┐
│       Agent Runner              │
│   · LLM API 调用               │
│   · 工具调用循环               │
│   · 上下文窗口管理             │
└────────────────────────────────┘
                │
    ┌───────────┼───────────┐
    │           │           │
┌───▼───┐   ┌───▼───┐   ┌───▼───┐   ┌───▼───┐
│Bash   │   │ Read  │   │ Write │   │ Memory │
│Tool   │   │ Tool  │   │ Tool  │   │ Search │
└───────┘   └───────┘   └───────┘   └────────┘
```

与 OpenClaw 的对应关系：

MiniClaw 组件	OpenClaw 对应	简化策略
Gateway	`src/gateway/server.ts`	去掉 HTTP 层、认证、服务发现
Agent Runner	`src/agents/pi-agent.ts`	去掉多模型回退、上下文修剪策略
Session Store	`src/sessions/`	纯内存 Map，不持久化
Channel Adapter	`src/channels/`	直接实现，不用插件架构
Tool System	`src/agents/tools/`	硬编码工具列表，不动态注册

42.1.3 技术选型建议

运行时与语言

选项	推荐	理由
语言	TypeScript	与 OpenClaw 一致，类型安全，生态丰富
运行时	Node.js 22+	原生 ES Modules、WebSocket 支持
包管理	pnpm	更快的安装速度，支持 workspace

核心依赖

```
{
  "dependencies": {
    "ws": "^8.0.0",                // WebSocket 服务器
    "openai": "^4.0.0",            // OpenAI API 客户端（兼容 Anthropic）
    "@anthropic-ai/sdk": "^0.30", // Anthropic API 客户端
    "grammy": "^1.0.0",            // Telegram Bot 框架
    "express": "^4.0.0",           // HTTP 服务器（WebChat）
    "zod": "^3.0.0",               // 运行时类型校验
    "chalk": "^5.0.0"              // 终端着色
  },
  "devDependencies": {
    "typescript": "^5.5.0",
    "vitest": "^2.0.0",
    "tsdown": "^0.5.0"
  }
}
```

LLM API 选择

MiniClaw 应支持至少两个 LLM 提供商：

1. **Anthropic Claude**：通过 `@anthropic-ai/sdk` ，支持工具调用、流式输出、思维过程
2. **OpenAI GPT**：通过 `openai` SDK，支持函数调用、流式输出

建议使用**适配器模式**（参考 OpenClaw 的 `src/providers/` ）统一不同提供商的 API 差异：

```typescript
interface LLMProvider {
  chat(params: {
    messages: Message[];
    tools?: ToolDefinition[];
    stream?: boolean;
  }): AsyncIterable<ChatChunk>;
}

class AnthropicProvider implements LLMProvider { /* ... */ }
class OpenAIProvider implements LLMProvider { /* ... */ }
```

项目目录结构

```
miniclaw/
├── src/
│   ├── gateway/
│   │   ├── server.ts        ← WebSocket 服务器
│   │   ├── protocol.ts      ← RPC 协议定义
│   │   └── session-store.ts ← 会话存储
│   ├── agent/
│   │   ├── runner.ts        ← Agent 运行循环
│   │   ├── context.ts       ← 上下文窗口管理
│   │   └── tools/
│   │       ├── bash.ts      ← Bash 工具
│   │       ├── read.ts      ← 文件读取工具
│   │       └── write.ts     ← 文件写入工具
│   ├── channels/
│   │   ├── telegram.ts      ← Telegram Bot 适配器
│   │   └── webchat.ts       ← WebChat 适配器
│   ├── memory/
│   │   └── markdown.ts      ← Markdown 记忆
│   ├── providers/
│   │   ├── anthropic.ts     ← Anthropic 适配器
│   │   └── openai.ts        ← OpenAI 适配器
│   └── config.ts            ← 配置加载
├── ui/                      ← WebChat 前端
├── package.json
├── tsconfig.json
└── vitest.config.ts
```

开发阶段规划

阶段	内容	预计工时
阶段 1	Gateway + Agent Runner + Bash 工具	8-12 小时
阶段 2	Telegram 通道 + WebChat	6-8 小时
阶段 3	文件工具 + 记忆系统	4-6 小时
阶段 4	Docker 部署 + 安全加固	4-6 小时
总计	完整 MVP	22-32 小时

对于有 TypeScript 和 Node.js 经验的学生，整个项目可以在一到两周内完成。

本节小结

1. **MiniClaw** 是 OpenClaw 核心设计的最小可行实现，聚焦于 Gateway + Agent + 通道 + 工具四大模块。
2. **架构设计** 借鉴 OpenClaw 的分层模式，但去掉了多 Agent、沙箱、Canvas 等高级功能。
3. **技术选型** 推荐 TypeScript + Node.js 22+，依赖 `ws`（WebSocket）、`openai` / `@anthropic-ai/sdk`（LLM）、`grammy`（Telegram）。
4. **适配器模式** 统一不同 LLM 提供商的 API 差异，便于后续扩展。
5. **四阶段开发计划**：Gateway → 通道 → 工具/记忆 → 部署，总计 22-32 小时。